Smart LMS Submission Management Component

Comprehensive Technical Documentation

System: Smart Learning Management System (Smart LMS)
Component: Submission Management, AI Feedback, Plagiarism Detection & Grading Subsystem
Developer: IT22586766 — Sachini Dilrangi
Date: April 2026
Version: 1.0

Table of Contents

1. System Overview
2. System Architecture
3. Database Design
4. Frontend Architecture
5. Text Submission System
6. AI Feedback Engine
7. Plagiarism Detection System
8. Grading System
9. Version Control System
10. Deadline Enforcement
11. Instructor Features
12. PDF Report Generation
13. Complete Data Flows & Workflows
14. Performance, Scalability & Security
15. Testing Strategy
16. System Integrations
17. Technical Challenges & Innovations
18. Limitations & Future Work
19. Technology Stack
20. API Reference

---

1. System Overview

1.1 Introduction

The Smart LMS Submission Management Component is a full-stack, microservice-based subsystem embedded within a larger collaborative Learning Management System. It handles the complete lifecycle of student text-based assignment submissions — from draft creation through real-time AI analysis, plagiarism detection, version snapshotting, and lecturer grading — all within a cohesive, responsive web interface.

Unlike conventional LMS submission modules that merely receive and store files, this component provides **live, in-editor intelligence**: as students compose their answers, the system continuously evaluates grammatical quality, semantic clarity, content completeness, topical relevance, and textual similarity against a corpus of peer submissions and internet content — all within the browser, with sub-5-second feedback latency.

1.2 Problem Statement

Traditional LMS tools suffer from several well-documented shortcomings in assessment workflows:

- **Delayed feedback**: Students receive evaluation only after deadlines, offering no opportunity to improve work in progress.
- **Manual plagiarism detection**: Instructors spend significant time cross-referencing submissions, a process that is both time-intensive and inconsistent.
- **Opaque grading**: Students receive numeric scores without structured diagnostic feedback tied to specific quality dimensions.
- **No version history**: Resubmissions overwrite originals, making it impossible to track student progress or detect post-deadline edits.
- **Disconnected tooling**: Separate systems for submission, feedback, and grading create fragmented student and instructor experiences.

This component addresses all five shortcomings in a single, integrated subsystem.

1.3 Component Scope

The component is responsible for:

Responsibility	Description
Assignment intake	Loading questions from the Projects & Tasks service
Draft management	Creating, updating, and locking submissions
Text answer persistence	Upsert-on-keystroke with debounce
Live AI feedback	Real-time grammar, clarity, completeness, relevance scoring
Live plagiarism detection	Peer-comparison + internet similarity on save
Version snapshotting	Immutable snapshots on formal submission
Deadline enforcement	Hard lock on all mutations after deadline
Lecturer grading	Per-question mark + feedback after deadline
PDF report generation	Downloadable feedback and plagiarism PDFs
Analytics dashboard	Submission statistics, grade distribution, plagiarism flags

1.4 Key Performance Targets

Metric	Target	Achieved
AI feedback latency (live)	< 5 s	~3–4 s (Zephyr-7B)
Plagiarism check latency (live)	< 3 s	~1–2 s (local peer) / ~3–4 s (internet)
Auto-save persistence delay	< 5 s after keystroke	5 s debounce
Submission lock after deadline	Immediate	Enforced at service layer
Version snapshot atomicity	100%	`@Transactional` + idempotency guard

---

2. System Architecture

2.1 High-Level Architecture

The system follows a **microservice architecture** with a Next.js 16 frontend acting as an orchestration layer. Four dedicated backend services handle distinct domains:

```
┌──────────────────────────────────────────────────────────────┐
│                    Next.js 16 Frontend (Port 3000)            │
│  React 19 · TypeScript · Tailwind CSS 4 · Lucide Icons        │
│                                                               |
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────────────┐  │
│  │ Student Pages│  │ Lecturer Pages│  │ Shared Hooks/Services │  │
│  └──────────────┘  └──────────────┘  └──────────────────────┘  │
└──────────────────────────────────────────────────────────────┘
         │              │              │              │
         ▼              ▼              ▼              ▼
  ┌──────────┐  ┌──────────────┐ ┌────────────┐ ┌──────────────┐
  │Submission │  │   Feedback   │ │ Integrity  │ │ Version Control│
  │Management │  │   Service    │ │ Monitoring │ │   Service      │
  │  :8081    │  │    :8083     │ │  Service   │ │   (in-process) │
  └─────┬────┘  └──────┬───────┘ │   :8084    │ └──────────────┘
        │              │         └─────┬──────┘
        ▼              ▼               ▼
  ┌──────────────────────────────────────────┐
  │           PostgreSQL :5432 / lms_db       │
  │  submission_schema · integrity_schema     │
  └──────────────────────────────────────────┘
```

2.2 Service Responsibilities

Submission Management Service (Port 8081)

The core service responsible for:

- CRUD operations on `Submission` and `Answer` entities
- Submission lifecycle state machine (DRAFT → SUBMITTED → GRADED)
- Deadline enforcement
- AI mark computation at submit time
- Aggregate score calculation
- Version snapshot orchestration
- Lecturer grading writes

Technology: Spring Boot 3.x, Spring Data JPA, PostgreSQL, Lombok

Feedback Service (Port 8083)

Stateless AI feedback service:

- Receives answer text + question text via REST
- Constructs a structured prompt for a hosted LLM (Zephyr-7B-Beta via HuggingFace/Featherless)
- Parses the LLM response into four numeric scores and three bullet-point arrays
- Applies consistency enforcement rules to eliminate internal contradictions
- Returns a structured `LiveFeedbackResponse` — no database writes

Technology: Spring Boot 3.x, HuggingFace Inference API, `@Async` with thread pool, Caffeine cache

Integrity Monitoring Service (Port 8084)

Dual-mode plagiarism detection:

- **Peer comparison**: TF-IDF cosine similarity + Jaccard + N-gram Dice against the class corpus
- **Internet similarity**: SerpAPI web search followed by corpus concatenation + TF-IDF cosine
- Risk score calculation and severity bucketing
- WebSocket push notifications on flag events
- Batch re-check and scheduled sweep
- PDF report generation

Technology: Spring Boot 3.x, Apache Commons Math, SerpAPI, WebSocket (STOMP), JasperReports/iText

Version Control (In-Process)

Originally planned as a separate service; integrated into the Submission Management Service to guarantee atomic transactional consistency between submission finalization and version snapshotting.

2.3 Inter-Service Communication

All communication is **synchronous HTTP REST** using Spring's `RestTemplate` for backend-to-backend calls and the browser's `fetch` API for frontend-to-backend calls.

There is no message queue or event bus in the current implementation — this is identified as a limitation in Section 18.

2.4 Authentication & Authorization

- **JWT-based authentication**: Tokens are issued by the Next.js auth service and stored in `localStorage['authToken']`.
- **Token structure**: `{ userId, userRole, ... }` signed with a shared secret.
- **Backend validation**: Each service validates the Bearer token in the `Authorization` header via `JwtUtils.extractRole()`, which checks both `role` and `userRole` claim keys for compatibility with multiple token issuers.
- **Role values**: `'student'` and `'lecture'` (note: not `'lecturer'`).

---

3. Database Design

3.1 Schema Overview

The component uses two PostgreSQL schemas within a shared `lms_db` database:

Schema	Owner Service	Purpose
`submission_schema`	Submission Management Service	Submissions, answers, version snapshots
`integrity_schema`	Integrity Monitoring Service	Plagiarism checks, similarity matches, internet matches

Hibernate `ddl-auto=update` manages all table DDL within each schema, ensuring schema evolution across deployments without manual migration scripts.

3.2 Submission Schema

Table: `submission_schema.submissions`

Column	Type	Constraints	Description
`id`	BIGINT	PK, auto-increment	Submission identifier
`assignment_id`	VARCHAR(255)	NOT NULL	Foreign key to assignment service
`student_id`	VARCHAR(255)	NOT NULL	Student identifier from JWT
`status`	VARCHAR(20)	NOT NULL	DRAFT / SUBMITTED / GRADED
`submission_type`	VARCHAR(20)		TEXT / FILE
`file_path`	TEXT		Path for file submissions
`word_count`	INT		Aggregate word count
`character_count`	INT		Aggregate character count
`submitted_at`	TIMESTAMP		When formally submitted
`deadline`	TIMESTAMP		Submission deadline
`ai_score`	DOUBLE		Aggregate AI quality score (0–100)
`plagiarism_score`	DOUBLE		Highest plagiarism similarity score
`lecturer_grade`	DOUBLE		Final grade set by lecturer
`lecturer_feedback`	TEXT		Aggregate lecturer feedback
`graded_at`	TIMESTAMP		When grading was completed
`question_marks_json`	TEXT		JSON: per-question marks from lecturer
`created_at`	TIMESTAMP		Creation timestamp (auto)
`updated_at`	TIMESTAMP		Last update timestamp (auto)

Indexes:

- `idx_submissions_student_assignment` on `(student_id, assignment_id)` — covers draft lookup
- `idx_submissions_assignment_status` on `(assignment_id, status)` — covers lecturer dashboard

Table: `submission_schema.answers`

Column	Type	Constraints	Description
`id`	BIGINT	PK, auto-increment	Answer identifier
`submission_id`	BIGINT	NOT NULL	FK to submissions.id
`student_id`	VARCHAR(255)		Student identifier (for cross-submission peer exclusion)

question_id	VARCHAR(255)	NOT NULL	Question identifier from assignment service
question_text	VARCHAR(2000)		Snapshot of question text at save time
answer_text	TEXT		Full answer content
word_count	INT		Word count from frontend
character_count	INT		Character count from frontend
last_modified	TIMESTAMP	NOT NULL	Auto-updated on every write
created_at	TIMESTAMP		Auto-set on first INSERT (nullable for migration)
grammar_score	DOUBLE		AI grammar score 0–10
clarity_score	DOUBLE		AI clarity score 0–10
completeness_score	DOUBLE		AI completeness score 0–10
relevance_score	DOUBLE		AI relevance score 0–10
ai_strengths	TEXT		Comma-separated strength bullets
ai_improvements	TEXT		Comma-separated improvement bullets
ai_suggestions	TEXT		Comma-separated suggestion bullets
feedback_saved_at	TIMESTAMP		When AI feedback was last persisted
similarity_score	DOUBLE		Plagiarism similarity 0–100
plagiarism_severity	VARCHAR(10)		LOW / MEDIUM / HIGH
plagiarism_flagged	BOOLEAN		True if severity >= MEDIUM
plagiarism_checked_at	TIMESTAMP		When plagiarism was last checked
ai_generated_mark	DOUBLE		Weighted AI mark (0–maxPoints), immutable after submit
lecturer_mark	DOUBLE		Lecturer-assigned mark
lecturer_feedback_text	TEXT		Lecturer feedback for this question
lecturer_updated_at	TIMESTAMP		When lecturer last updated
lecturer_updated_by	VARCHAR(100)		Lecturer ID who updated

Indexes:

- `idx_answers_submission_id_question_id` on `(submission_id, question_id)` — covers upsert lookup
- `idx_answers_question_id_last_modified` on `(question_id, last_modified)` — covers peer plagiarism query

Table: `submission_schema.submission_versions`

Stores immutable snapshots created on each formal submission:

Column	Type	Description
`id`	BIGINT	PK
`submission_id`	BIGINT	FK to submissions
`version_number`	INT	Sequential version (1, 2, 3, ...)
`answers_snapshot_json`	TEXT	Full JSON snapshot of all answers at submit time
`total_word_count`	INT	Total words at this version
`ai_score_snapshot`	DOUBLE	AI score at this version
`created_at`	TIMESTAMP	When snapshot was created

3.3 Integrity Schema

Table: `integrity_schema.plagiarism_checks`

Column	Type	Description
`id`	BIGINT	PK
`submission_id`	VARCHAR	Submission being checked
`student_id`	VARCHAR	Student identifier
`assignment_id`	VARCHAR	Assignment identifier
`content_type`	VARCHAR(10)	TEXT / CODE
`check_type`	VARCHAR(20)	REALTIME / BATCH / SCHEDULED
`status`	VARCHAR(20)	PENDING / COMPLETED / FAILED
`max_similarity_score`	DOUBLE	Highest similarity found (0.0–1.0)
`risk_score`	DOUBLE	Computed risk (0–100)
`severity`	VARCHAR(10)	LOW / MEDIUM / HIGH
`flagged`	BOOLEAN	Whether this check triggered a flag
`question_id`	VARCHAR	Question checked (for per-question checks)
`internet_similarity`	DOUBLE	Highest internet match score
`metadata`	JSONB	Check metadata (thresholds, options)
`created_at`	TIMESTAMP	Check creation time
`completed_at`	TIMESTAMP	When check finished
`checked_by`	VARCHAR	Who triggered the check (SYSTEM / lecturerId)

Table: `integrity_schema.similarity_matches`

Per-peer match records linked to a plagiarism check:

Column	Type	Description
`id`	BIGINT	PK
`check_id`	BIGINT	FK to plagiarism_checks
`matched_submission_id`	VARCHAR	The peer submission matched against
`matched_student_id`	VARCHAR	The peer student
`similarity_score`	DOUBLE	Similarity score for this pair
`algorithm`	VARCHAR	Algorithm that detected this match

Table: `integrity_schema.internet_matches`

Per-URL internet match records:

Column	Type	Description
`id`	BIGINT	PK
`check_id`	BIGINT	FK to plagiarism_checks
`url`	TEXT	Matched URL
`title`	VARCHAR	Page title
`snippet`	TEXT	Matched text snippet
`similarity_score`	DOUBLE	Similarity to this page
`source_category`	VARCHAR	ACADEMIC / ENCYCLOPEDIA / NEWS / BLOG / etc.

---

4. Frontend Architecture

4.1 Technology Stack

Layer	Technology	Version
Framework	Next.js	16
UI Library	React	19
Language	TypeScript	5.x
Styling	Tailwind CSS	4
Icons	Lucide React	Latest
State Management	React hooks (no Redux)	—

HTTP Client	Browser `fetch` API	—

4.2 Directory Structure

```
frontend/
├── app/
│   └── submissions/
│       ├── student/
│       │   ├── answer/[assignmentId]/page.tsx      # Main text-answer page
│       │   ├── submit/page.tsx                     # File submission page
│       │   ├── my-submissions/page.tsx             # Submission list
│       │   ├── feedback/[id]/page.tsx              # Feedback view
│       │   └── version-history/[id]/page.tsx       # Version history
│       └── lecturer/
│           ├── submissions/page.tsx                # Submission management
│           ├── grading/[submissionId]/page.tsx     # Per-submission grading
│           └── plagiarism/page.tsx                 # Plagiarism dashboard
├── components/submissions/
│   ├── QuestionCard.tsx                            # Wires editor + feedback + plagiarism
│   ├── RichTextEditor.tsx                          # Auto-resize textarea + word count
│   ├── LiveFeedbackPanel.tsx                       # Real-time AI scores + bullets
│   ├── PlagiarismWarning.tsx                       # Severity chip below editor
│   ├── AIFeedbackCard.tsx                          # Static feedback display
│   ├── PlagiarismReportCard.tsx                    # Traffic-light + match list
│   ├── VersionTimeline.tsx                         # Version history timeline
│   ├── DiffViewer.tsx                              # Line-level diff view
│   └── FileUploader.tsx                            # Drag-and-drop file upload
├── hooks/
│   ├── useAnswerEditor.ts                          # Per-question debounce logic
│   ├── useSubmissions.ts                           # Submission CRUD hooks
│   ├── useVersions.ts                              # Version history hooks
│   ├── useFeedback.ts                              # Feedback hooks
│   ├── usePlagiarism.ts                            # Plagiarism hooks
│   └── useModuleConfig.ts                          # Module-level configuration
├── lib/api/
│   └── submission-services.ts                     # All API call functions
└── types/
    └── submission.types.ts                         # All TypeScript interfaces
```

4.3 Core Hook: `useAnswerEditor`

The `useAnswerEditor` hook is the central orchestration point for the student answer editing experience. It manages three concurrent, independent debounce timers using `useRef` (not `setTimeout` directly, to prevent stale closures and allow cancellation):

```typescript
// Simplified structure
function useAnswerEditor(submissionId: string, questionId: string, questionText: string) {
  const [text, setText] = useState('');
  const [liveFeedback, setLiveFeedback] = useState<LiveFeedback | null>(null);
  const [livePlagiarism, setLivePlagiarism] = useState<LivePlagiarismResult | null>(null);
```

```
  const [saveStatus, setSaveStatus] = useState<'idle' | 'saving' | 'saved' | 'error'>
('idle');

  const feedbackTimerRef = useRef<NodeJS.Timeout | null>(null);
  const plagiarismTimerRef = useRef<NodeJS.Timeout | null>(null);
  const autoSaveTimerRef = useRef<NodeJS.Timeout | null>(null);

  const handleTextChange = (newText: string) => {
    setText(newText);

    // Reset all timers on each keystroke
    clearTimeout(feedbackTimerRef.current!);
    clearTimeout(plagiarismTimerRef.current!);
    clearTimeout(autoSaveTimerRef.current!);

    feedbackTimerRef.current = setTimeout(() => triggerLiveFeedback(newText), 3000);
    plagiarismTimerRef.current = setTimeout(() => triggerLivePlagiarism(newText), 3000);
    autoSaveTimerRef.current = setTimeout(() => saveAnswer(newText), 5000);
  };

  return { text, handleTextChange, liveFeedback, livePlagiarism, saveStatus };
}
```

Debounce intervals:

Operation	Delay	Rationale
AI Feedback	3 seconds	Balances responsiveness vs. LLM API cost
Plagiarism Check	3 seconds	Network + computation cost
Auto-save	5 seconds	Avoids excessive DB writes on fast typists

4.4 TypeScript Type System

The component defines a comprehensive type system in `submission.types.ts` covering all entities:

```
interface Submission { id: string; assignmentId: string; studentId: string; status: 'DRAFT'
| 'SUBMITTED' | 'GRADED'; ... }
interface Answer { id: number; submissionId: string; questionId: string; answerText: string;
wordCount: number; grammarScore?: number; clarityScore?: number; completenessScore?: number;
relevanceScore?: number; strengths?: string[]; improvements?: string[]; suggestions?:
string[]; similarityScore?: number; plagiarismSeverity?: 'LOW' | 'MEDIUM' | 'HIGH';
plagiarismFlagged?: boolean; aiGeneratedMark?: number; lecturerMark?: number; ... }
interface LiveFeedback { grammarScore: number; clarityScore: number; completenessScore:
number; relevanceScore: number; strengths: string[]; improvements: string[]; suggestions:
string[]; }
interface LivePlagiarismResult { similarityScore: number; severity: 'LOW' | 'MEDIUM' |
'HIGH'; flagged: boolean; matchedStudents?: string[]; internetMatches?: InternetMatch[]; }
```

---

5. Text Submission System

5.1 Question Loading

When a student navigates to `/submissions/student/answer/[assignmentId]`, the frontend:

1. Decodes the JWT to extract `studentId` and `userRole`
2. Calls `getAssignmentWithFallback(assignmentId)` which attempts:
   - `GET /api/projects-and-tasks/project/[id]` → maps `mainTasks[]` to `Question[]`
   - `GET /api/projects-and-tasks/task/[id]` → maps `subtasks[]` to `Question[]`
   - `GET http://localhost:8081/api/assignments/[id]` as legacy fallback
3. Checks for an existing draft via `GET /api/submissions/student/{studentId}?assignmentId={id}&status=DRAFT`
4. Creates a new draft via `POST /api/submissions` if none exists
5. Loads existing answers via `GET /api/submissions/{submissionId}/answers`

5.2 Draft State Machine

```
                POST /api/submissions
               (assignmentId, studentId)
                       │
                       ▼
                  ┌─────────┐
                  │  DRAFT  │ ◄── Auto-save (every 5s after keystroke)
                  └─────────┘      PUT /api/submissions/{id}/answers/{qId}
                       │
                       │
          Student clicks "Submit Assignment"
                       │
                       ▼
          Deadline check (service layer)
          AI mark computation
          Version snapshot (atomic)
                       │
                       ▼
                 ┌──────────┐
                 │ SUBMITTED │ ◄── Immutable (no further edits)
                 └──────────┘
                       │
                       │
          Lecturer grades all questions
                       │
                       ▼
                 ┌────────┐
                 │ GRADED │
                 └────────┘
```

5.3 Answer Upsert Pattern

The backend implements a safe upsert to handle auto-save correctly:

```java
// AnswerService.java
public AnswerResponse saveOrUpdateAnswer(Long submissionId, String questionId,
```

```
SaveAnswerRequest req) {
    Answer answer = answerRepository
        .findBySubmissionIdAndQuestionId(submissionId, questionId)
        .orElseGet(() -> Answer.builder()
            .submissionId(submissionId)
            .questionId(questionId)
            .build());

    answer.setAnswerText(req.getAnswerText());
    answer.setQuestionText(req.getQuestionText());
    answer.setWordCount(req.getWordCount());
    answer.setCharacterCount(req.getCharacterCount());
    answer.setStudentId(req.getStudentId());

    return toResponse(answerRepository.save(answer));
}
```

This pattern guarantees exactly one `Answer` row per `(submissionId, questionId)` pair regardless of how many auto-save calls fire concurrently.

5.4 Minimum Word Enforcement

The submission service enforces a minimum word count before accepting a formal submission:

```
// Configurable via application.properties
@Value("${submission.min-words-per-answer:3}")
private int minWordsPerAnswer;

// At submit time:
for (Answer answer : answers) {
    if (answer.getWordCount() < minWordsPerAnswer) {
        throw new ValidationException("Answer to question '" + answer.getQuestionText() +
            "' must contain at least " + minWordsPerAnswer + " words.");
    }
}
```

5.5 Real-Time UI Feedback

The `QuestionCard` component renders a two-column layout:

```
┌──────────────────────────────┬──────────────────────────────┐
│         RichTextEditor       │       LiveFeedbackPanel      │
│  ┌──────────────────────┐    │  ┌──────────────────────┐    │
│  │ [Auto-resize textarea]    │  │ Grammar    ▓▓▓▓▓▓▓▒▒  8.2 │  │
│  │                      │    │  │ Clarity    ▓▓▓▓▒▒▒▒▒  6.0 │  │
│  │                      │    │  │ Completeness ▓▓▓▓▓▓▓▒  8.5│  │
│  └──────────────────────┘    │  │ Relevance  ▓▓▓▓▓▓▓▒▒  9.1 │  │
│  Word count: 142 / 500 [green]  ├──────────────────────┤    │
│                              │  │ ✓ Strengths: ...     │    │
│  [PlagiarismWarning chip]    │  │ ⚠ Improvements: ...  │    │
```

```
│  MEDIUM  72% similarity         │  │ → Suggestions: ...         │  │
└──────────────────────────────────┴──────────────────────────────────┘
```

The word count bar colour changes dynamically:

- Green: word count ≥ 80% of target
- Amber: 40–80% of target
- Red: < 40% of target

---

6. AI Feedback Engine

6.1 Architecture

The Feedback Service is a **stateless** Spring Boot application. It:

1. Receives a `POST /api/feedback/live` request with `{ questionText, answerText, studentId }`
2. Performs gibberish detection
3. Constructs a structured LLM prompt
4. Calls the HuggingFace Inference API (Featherless routing to Zephyr-7B-Beta)
5. Parses and validates the LLM response
6. Returns a structured `LiveFeedbackResponse`

No database writes occur in this service. Persistence of feedback scores is handled by the Submission Management Service via `PATCH /api/submissions/{id}/answers/{qId}/analysis`.

6.2 Gibberish Detection

Before invoking the LLM, the service checks whether the answer is meaningful:

```java
private boolean isGibberish(String text) {
    if (text == null || text.trim().length() < 10) return true;

    String[] words = text.toLowerCase().split("\\s+");
    int flaggedCount = 0;

    for (String word : words) {
        // Flag words with excessive consecutive consonants (> 4)
        if (word.matches(".*[^aeiou]{5,}.*")) flaggedCount++;
        // Flag words with excessive consecutive vowels (> 3)
        if (word.matches(".*[aeiou]{4,}.*")) flaggedCount++;
        // Flag very short non-words (< 2 chars)
        if (word.length() < 2 && !word.matches("[ai]")) flaggedCount++;
    }

    // If more than 40% of words are flagged, consider it gibberish
    return (double) flaggedCount / words.length > 0.40;
}
```

If gibberish is detected, the service returns a fixed response indicating the answer requires revision.

6.3 Prompt Engineering

The service uses different prompt templates for short and long answers:

Short answer prompt (< 100 words):

```
You are an educational assessment assistant. Evaluate this student answer briefly.

Question: {questionText}
Student Answer: {answerText}

Rate ONLY these 4 dimensions (0.0-10.0 scale, one decimal):
- GRAMMAR: spelling, punctuation, sentence structure
- CLARITY: how clear and well-expressed the answer is
- COMPLETENESS: how thoroughly the question is answered
- RELEVANCE: how on-topic and accurate the answer is

Then provide:
- STRENGTHS: 2 brief bullet points (start each with "- ")
- IMPROVEMENTS: 2 brief bullet points (start each with "- ")
- SUGGESTIONS: 1 specific actionable suggestion (start with "- ")

Format EXACTLY:
GRAMMAR: X.X
CLARITY: X.X
COMPLETENESS: X.X
RELEVANCE: X.X
STRENGTHS:
- point1
- point2
IMPROVEMENTS:
- point1
- point2
SUGGESTIONS:
- suggestion1
```

Long answer prompt (≥ 100 words): Adds context about expected depth and nuance, and requests 3 bullets per category.

6.4 Response Parsing

The LLM response is parsed using regex patterns for each field:

```java
private static final Pattern GRAMMAR_PATTERN = Pattern.compile("GRAMMAR:\\s*([0-9](?:\\.[0-9])?)");
private static final Pattern CLARITY_PATTERN = Pattern.compile("CLARITY:\\s*([0-9](?:\\.[0-9])?)");
// etc.

private Double extractScore(String response, Pattern pattern) {
    Matcher m = pattern.matcher(response);
    if (m.find()) {
        double val = Double.parseDouble(m.group(1));
        return Math.max(0.0, Math.min(10.0, val)); // Clamp to [0, 10]
    }
```

```
    return null; // Signal that this score was not found
}
```

Bullet points are extracted by splitting on newlines and filtering for lines that start with `-`.

6.5 Consistency Enforcement

After parsing, three consistency rules are applied to eliminate internal contradictions in the LLM output:

Rule A — High score / negative language mismatch: If a score ≥ 8.0 but the corresponding bullet point contains highly negative words (e.g., "incorrect", "wrong", "poor", "missing"), the score is capped at 7.5 and a note is added to improvements.

Rule B — Low score / positive language mismatch: If a score ≤ 3.0 but all bullet points for that dimension are positive (contain "good", "well", "clear", "excellent"), the score is raised to 4.0 minimum.

Rule C — Dimension agreement: If grammar score and clarity score differ by more than 4.0 points, the gap is reduced by averaging toward the mean. Extreme divergence typically indicates a parsing artefact.

```
private static final List<String> POSITIVE_SIGNAL_WORDS = List.of(
    "excellent", "good", "well", "clear", "strong", "effective", "thorough",
    "accurate", "precise", "detailed", "logical", "organized", "coherent"
);

private void applyConsistencyRules(LiveFeedbackResponse response) {
    // Rule A
    if (response.getGrammarScore() >= 8.0 &&
containsNegativeSignals(response.getImprovements())) {
        response.setGrammarScore(Math.min(response.getGrammarScore(), 7.5));
    }
    // Rule B, Rule C ...
}
```

6.6 Asynchronous Processing

The live feedback endpoint is decorated with `@Async("feedbackTaskExecutor")`, allowing multiple concurrent feedback requests without blocking Tomcat worker threads:

```
@Bean("feedbackTaskExecutor")
public Executor feedbackTaskExecutor() {
    ThreadPoolTaskExecutor executor = new ThreadPoolTaskExecutor();
    executor.setCorePoolSize(5);
    executor.setMaxPoolSize(20);
    executor.setQueueCapacity(50);
    executor.setThreadNamePrefix("FeedbackAsync-");
    executor.initialize();
    return executor;
}
```

The controller returns `CompletableFuture<ResponseEntity<ApiResponse<LiveFeedbackResponse>>>`, which Spring MVC handles as a deferred result, freeing the thread immediately.

6.7 Caching

Feedback for identical `(questionText, answerText)` pairs is cached for 10 minutes using Caffeine:

```
@Cacheable(value = "feedbackCache", key = "#request.questionText + ':' +
#request.answerText.hashCode()")
public CompletableFuture<ApiResponse<LiveFeedbackResponse>>
generateLiveFeedback(LiveFeedbackRequest request) { ... }
```

This significantly reduces LLM API calls for students who pause and resume typing on the same content.

---

7. Plagiarism Detection System

7.1 Detection Modes

The system operates in two modes triggered at different points in the workflow:

Mode	Trigger	Algorithms	Scope
Real-time	Auto-save debounce (3s)	Peer (4 algos) + Internet	Current assignment's answers
Batch	Lecturer-triggered recheck	Peer (4 algos) + Internet	All submissions in assignment
Scheduled	Nightly sweep (configurable)	Peer (4 algos)	All active assignments

7.2 Text Similarity Algorithms

The service implements four distinct text similarity algorithms. For each pair of texts being compared, all four are computed and the maximum is taken as the final similarity score.

Algorithm 1: TF-IDF Cosine Similarity

TF-IDF (Term Frequency–Inverse Document Frequency) weighs terms by how often they appear in the document relative to the entire corpus, then computes the cosine of the angle between the two resulting vectors.

Term Frequency:

```
TF(t, d) = count(t in d) / total_terms(d)
```

Inverse Document Frequency:

```
IDF(t, D) = log(|D| / |{d ∈ D : t ∈ d}|)
```

TF-IDF weight:

```
TFIDF(t, d, D) = TF(t, d) × IDF(t, D)
```

Cosine similarity:

```
cosine(A, B) = (A · B) / (||A|| × ||B||)
```

Implementation using Apache Commons Math:

```java
private double tfidfCosineSimilarity(String text1, String text2) {
    Map<String, Double> tfidf1 = computeTfIdf(text1, Arrays.asList(text1, text2));
    Map<String, Double> tfidf2 = computeTfIdf(text2, Arrays.asList(text1, text2));
    Set<String> vocab = new HashSet<>(tfidf1.keySet());
    vocab.addAll(tfidf2.keySet());

    double[] v1 = vocab.stream().mapToDouble(t -> tfidf1.getOrDefault(t, 0.0)).toArray();
    double[] v2 = vocab.stream().mapToDouble(t -> tfidf2.getOrDefault(t, 0.0)).toArray();

    ArrayRealVector vec1 = new ArrayRealVector(v1);
    ArrayRealVector vec2 = new ArrayRealVector(v2);
    double norm = vec1.getNorm() * vec2.getNorm();
    return norm == 0 ? 0.0 : vec1.dotProduct(vec2) / norm;
}
```

Algorithm 2: Jaccard Similarity

Jaccard similarity measures the overlap of unique word sets between two documents:

```
Jaccard(A, B) = |A ∩ B| / |A ∪ B|
```

Where A and B are the sets of unique words in each text (after stop-word removal).

```java
private double jaccardSimilarity(String text1, String text2) {
    Set<String> set1 = tokenize(text1);
    Set<String> set2 = tokenize(text2);
    Set<String> intersection = new HashSet<>(set1);
    intersection.retainAll(set2);
    Set<String> union = new HashSet<>(set1);
    union.addAll(set2);
    return union.isEmpty() ? 0.0 : (double) intersection.size() / union.size();
}
```

Algorithm 3 & 4: N-gram Dice Similarity (n=3, n=5)

N-gram Dice similarity computes the overlap of character n-gram sets using the Sørensen–Dice coefficient:

```
Dice(A, B) = 2 × |ngrams(A) ∩ ngrams(B)| / (|ngrams(A)| + |ngrams(B)|)
```

N-grams of size 3 catch short phrase similarities; n-grams of size 5 catch longer phrase similarities. Using both reduces sensitivity to parameter choice.

```java
private double ngramDiceSimilarity(String text1, String text2, int n) {
    Set<String> ngrams1 = extractNgrams(text1, n);
    Set<String> ngrams2 = extractNgrams(text2, n);
    Set<String> intersection = new HashSet<>(ngrams1);
    intersection.retainAll(ngrams2);
    int total = ngrams1.size() + ngrams2.size();
    return total == 0 ? 0.0 : (2.0 * intersection.size()) / total;
}
```

Ensemble Strategy

```
public double calculateBestSimilarity(String text1, String text2) {
    double tfidf   = tfidfCosineSimilarity(text1, text2);
    double jaccard = jaccardSimilarity(text1, text2);
    double dice3   = ngramDiceSimilarity(text1, text2, 3);
    double dice5   = ngramDiceSimilarity(text1, text2, 5);
    return Math.max(Math.max(tfidf, jaccard), Math.max(dice3, dice5));
}
```

Taking the maximum rather than the average ensures the system catches plagiarism detected by any one of the four methods, even if others disagree.

7.3 Internet Similarity

For internet similarity, the service uses a **corpus concatenation** strategy:

1. Send the answer text as a search query to SerpAPI (Google Search API)
2. Retrieve up to 3 search results
3. Concatenate all result snippets into a single "internet corpus" string
4. Run TF-IDF cosine similarity between the answer and the corpus

```
public double calculateInternetSimilarity(String answerText) {
    List<SearchResult> results = serpApiService.search(answerText, 3);
    if (results.isEmpty()) return 0.0;

    String corpus = results.stream()
        .map(r -> r.getSnippet() + " " + r.getTitle())
        .collect(Collectors.joining(" "));

    return tfidfCosineSimilarity(answerText, corpus);
}
```

This approach is more robust than per-snippet comparison because it avoids false positives from short snippets that may coincidentally share common vocabulary.

7.4 Risk Score Calculation

After similarity scores are computed, a risk score is calculated:

```
double riskScore = maxPeerSimilarity * 100 + Math.min(numHighMatches * 5, 20);
```

Where `numHighMatches` is the count of peer answers with similarity > 0.45.

The risk score maps to severity buckets:

Risk Score	Severity	Flagged
< 30	LOW	false
30–60	MEDIUM	true

> 60	HIGH	true

7.5 Peer Comparison Scope

For efficiency, peer comparison queries at most 200 recent answers for the same question, ordered by `last_modified DESC`:

```
List<Answer> peers = answerRepository.findByQuestionIdOrderByLastModifiedDesc(questionId)
    .stream()
    .filter(a -> !a.getStudentId().equals(currentStudentId)) // Exclude self
    .limit(200)
    .collect(Collectors.toList());
```

The student's own answers (across all submissions and versions) are excluded via the `student_id` column on `Answer`.

7.6 WebSocket Notifications

When a real-time check produces a HIGH severity result, the service pushes an immediate WebSocket notification to the student's browser:

```
messagingTemplate.convertAndSend(
    "/topic/plagiarism-warnings/" + sessionId,
    PlagiarismWarningMessage.builder()
        .severity("HIGH")
        .similarityScore(riskScore)
        .message("High similarity detected. Please revise your answer.")
        .build()
);
```

The student frontend subscribes to this topic when the answer page loads.

7.7 Severity Display in the UI

The `PlagiarismWarning` component renders a severity chip below each answer editor:

- **LOW** (< 30): Grey chip, no warning text
- **MEDIUM** (30–60): Amber chip with "Moderate similarity detected"
- **HIGH** (> 60): Red chip with "High similarity — please revise"

---

8. Grading System

8.1 Grading Architecture

Grading is a two-stage process:

1. **AI-suggested mark** (computed at submit time, immutable)
2. **Lecturer mark** (set after deadline, overrides AI suggestion)

Both are stored per-question in the `answers` table, giving full traceability.

8.2 AI Mark Computation

At submit time, for each answer that has AI feedback scores, a weighted mark is computed:

```
ai_generated_mark = (relevanceScore × 0.40 +
                     completenessScore × 0.30 +
                     clarityScore × 0.15 +
                     grammarScore × 0.15) × (maxPoints / 10.0)
```

Where `maxPoints` defaults to 10 but can be configured per-assignment.

Handling missing scores:

If some dimensions are null (AI feedback never ran for this answer), the formula is renormalized using only the available dimensions:

```
double totalWeight = 0.0, weightedSum = 0.0;
if (relevance != null) { weightedSum += relevance * 0.40; totalWeight += 0.40; }
if (completeness != null) { weightedSum += completeness * 0.30; totalWeight += 0.30; }
if (clarity != null) { weightedSum += clarity * 0.15; totalWeight += 0.15; }
if (grammar != null) { weightedSum += grammar * 0.15; totalWeight += 0.15; }

double aiMark = (totalWeight > 0) ? (weightedSum / totalWeight) * (maxPoints / 10.0) : null;
```

Worked Example:

Given: grammar=7.2, clarity=6.5, completeness=8.1, relevance=9.0, maxPoints=10

```
ai_mark = (9.0×0.40 + 8.1×0.30 + 6.5×0.15 + 7.2×0.15) × (10/10)
        = (3.60 + 2.43 + 0.975 + 1.08) × 1.0
        = 8.085
        ≈ 8.1/10
```

8.3 Aggregate Submission Score

After per-question AI marks are computed, an aggregate submission-level AI score is calculated:

```
double totalMark = answers.stream()
    .filter(a -> a.getAiGeneratedMark() != null)
    .mapToDouble(Answer::getAiGeneratedMark)
    .sum();

double maxPossible = answers.size() * maxPointsPerQuestion;
double aiScore = (maxPossible > 0) ? (totalMark / maxPossible) * 100.0 : null;
submission.setAiScore(aiScore); // 0–100 scale
```

8.4 Lecturer Grading Workflow

After the submission deadline:

1. Lecturer opens the grading page at `/submissions/lecturer/grading/{submissionId}`
2. The page loads all questions with the student's answer text and the AI-suggested mark pre-populated

3. The lecturer can override the mark and add feedback text per question
4. On save, `PATCH /api/submissions/{id}/answers/{qId}/analysis` is called with `{ lecturerMark, lecturerFeedbackText }`
5. After all questions are graded, the lecturer sets a final grade via `POST /api/submissions/{id}/grade`
6. The submission status transitions to GRADED

8.5 Dual-Path Grade Persistence

Lecturer marks are written to two locations for completeness:

1. **Working copy** (`answers` table): `lecturer_mark` , `lecturer_feedback_text` , `lecturer_updated_at` , `lecturer_updated_by`
2. **Latest version snapshot** (`submission_versions` table): The `answers_snapshot_json` is updated for the most recent version to include lecturer marks alongside the answer text

This ensures both the live answers view and historical version reports show consistent grading.

---

9. Version Control System

9.1 Versioning Strategy

Each time a student formally submits an assignment, an immutable version snapshot is created. This enables:

- Tracking improvement across resubmissions (if allowed)
- Detecting post-deadline edits (snapshots are immutable)
- Generating version-specific plagiarism and feedback reports
- Displaying a visual diff between any two versions

9.2 Snapshot Creation

Version snapshots are created within the same `@Transactional` context as the submission state change:

```
@Transactional
public Submission submitAssignment(Long submissionId, String studentId) {
    Submission submission = getSubmission(submissionId);
    validateDeadline(submission);
    computeAiMarks(submission);

    submission.setStatus("SUBMITTED");
    submission.setSubmittedAt(LocalDateTime.now());
    Submission saved = submissionRepository.save(submission);

    // Atomic snapshot within same transaction
    createVersionSnapshot(saved);

    return saved;
}

private void createVersionSnapshot(Submission submission) {
    int nextVersion = versionRepository
        .findMaxVersionNumberBySubmissionId(submission.getId())
        .map(v -> v + 1)
        .orElse(1);
```

```
    // Idempotency guard
    if (versionRepository.existsBySubmissionIdAndVersionNumber(submission.getId(),
nextVersion)) {
        return; // Already created (retry/duplicate call)
    }

    List<Answer> answers = answerRepository.findBySubmissionId(submission.getId());
    String snapshot = objectMapper.writeValueAsString(answers);

    SubmissionVersion version = SubmissionVersion.builder()
        .submissionId(submission.getId())
        .versionNumber(nextVersion)
        .answersSnapshotJson(snapshot)
        .totalWordCount(answers.stream().mapToInt(a -> a.getWordCount() != null ?
a.getWordCount() : 0).sum())
        .aiScoreSnapshot(submission.getAiScore())
        .build();

    versionRepository.save(version);
}
```

9.3 Diff Algorithm

The `DiffViewer` frontend component implements a line-level diff algorithm:

1. Split both version's answer texts into lines
2. Compute the Longest Common Subsequence (LCS) using dynamic programming
3. Mark lines as `ADDED` (only in new), `REMOVED` (only in old), or `UNCHANGED`
4. Render with colour coding: green for added, red for removed, grey for unchanged

Long unchanged blocks are collapsible to focus attention on changes.

9.4 Version Timeline UI

The `VersionTimeline` component renders a vertical timeline:

```
● Version 3 — Submitted 15 Mar 2026, 14:22  [AI Score: 81%]
│   [Compare] checkbox
│
● Version 2 — Submitted 12 Mar 2026, 09:11  [AI Score: 74%]
│   [Compare] checkbox ← select two to diff
│
● Version 1 — Submitted 10 Mar 2026, 16:45  [AI Score: 68%]
    [Compare] checkbox
```

When two checkboxes are selected, a "Compare Versions" button appears, navigating to the diff view.

---

10. Deadline Enforcement

10.1 Enforcement Points

Deadline enforcement is applied at the service layer (not the database layer or frontend) to ensure it cannot be bypassed by direct API calls:

Operation	Deadline Check
Formal submission	Submission must be BEFORE deadline
Answer auto-save	PERMITTED after deadline (student can still save drafts locally)
AI feedback call	PERMITTED after deadline (stateless, no DB write)
Plagiarism check	PERMITTED after deadline (read-only analysis)
Lecturer grading	ONLY permitted after deadline

Note: The submission service does NOT prevent auto-saves after the deadline. This is intentional — it prevents data loss if a student's browser is still open when the deadline passes. However, the "Submit" action is blocked.

10.2 Deadline Validation Logic

```
private void validateDeadline(Submission submission) {
    if (submission.getDeadline() == null) return; // No deadline set = always open

    LocalDateTime now = LocalDateTime.now();
    if (now.isAfter(submission.getDeadline())) {
        throw new DeadlinePassedException(
            "Submission deadline was " + submission.getDeadline() +
            ". Current time: " + now
        );
    }
}
```

10.3 Frontend Deadline Countdown

The student answer page displays a live countdown timer using `setInterval`:

```
// DeadlineCountdown component
const [timeLeft, setTimeLeft] = useState<string>('');
useEffect(() => {
    const interval = setInterval(() => {
        const diff = deadline.getTime() - Date.now();
        if (diff <= 0) { setTimeLeft('Deadline passed'); clearInterval(interval); return; }
        const h = Math.floor(diff / 3600000);
        const m = Math.floor((diff % 3600000) / 60000);
        const s = Math.floor((diff % 60000) / 1000);
        setTimeLeft(`${h}h ${m}m ${s}s remaining`);
    }, 1000);
    return () => clearInterval(interval);
}, [deadline]);
```

The colour of the countdown changes:

- Green: > 1 hour remaining
- Amber: 15 minutes to 1 hour remaining
- Red: < 15 minutes remaining

---

11. Instructor Features

11.1 Submission Management Dashboard

The lecturer's submissions page (`/submissions/lecturer/submissions`) provides:

- **Submission table**: All submissions for the lecturer's assignments, with status badges (DRAFT / SUBMITTED / GRADED), student name, submission time, AI score, and plagiarism score
- **Filter controls**: By assignment, status, and date range
- **Sort options**: By submission time, AI score, plagiarism score, student name
- **Bulk re-check**: Select multiple submissions and trigger batch plagiarism recheck
- **Quick grade access**: Click any submission to open the grading view

11.2 Grading Interface

The grading page (`/submissions/lecturer/grading/{submissionId}`) shows:

For each question:

1. Question text (from snapshot)
2. Student's answer text (full, scrollable)
3. AI-suggested mark (pre-populated, greyed out) with breakdown tooltip
4. AI feedback bullets (strengths / improvements / suggestions)
5. Plagiarism severity chip
6. Input: Lecturer mark (numeric, 0–maxPoints)
7. Textarea: Lecturer feedback text
8. "Save Question" button

At the bottom:

- Summary of all question marks
- Final grade input (can differ from sum of question marks for holistic adjustment)
- "Finalize Grade" button → transitions submission to GRADED

11.3 Plagiarism Dashboard

The plagiarism dashboard (`/submissions/lecturer/plagiarism`) shows:

- **Assignment-level statistics**: Total submissions, flagged count, percentage flagged, score distribution histogram
- **Flagged submissions list**: Sorted by risk score (highest first)
- **Per-submission card**: Student name, AI score, similarity score, severity badge, internet match URLs, peer match table
- **Recheck controls**: Per-submission recheck button and bulk "Recheck All" button

11.4 Bulk Plagiarism Recheck

The bulk recheck feature processes submissions sequentially (not in parallel) to avoid overwhelming the SerpAPI rate limits:

```
// Sequential to avoid thundering-herd on SerpAPI
for (const submissionId of selectedSubmissionIds) {
    await recheckPlagiarism(submissionId);
    setProgress(prev => prev + 1);
}
```

Progress is displayed in a modal with a progress bar.

11.5 Analytics Overview

The module overview page shows aggregate statistics:

- Total submissions this period
- Submissions requiring grading
- Flagged plagiarism count (loaded from `GET /api/integrity/checks/flagged`)
- Grade distribution chart
- Recent activity feed

---

12. PDF Report Generation

12.1 Available Reports

Two PDF reports can be generated:

Report	Endpoint	Contents
Feedback Report	`GET /api/integrity/reports/{submissionId}/feedback`	All questions, answers, AI scores, per-dimension breakdowns, strengths/improvements/suggestions
Plagiarism Report	`GET /api/integrity/reports/{submissionId}/plagiarism`	Similarity scores, peer matches (anonymized), internet matches with URLs, risk score, severity

12.2 Generation Architecture

PDF generation is asynchronous. When the endpoint is called:

1. A generation job is queued
2. The API returns `202 Accepted` with a job ID
3. The client polls `GET /api/integrity/reports/status/{jobId}` every 2 seconds
4. When complete, the API returns `200 OK` with a download URL
5. The frontend redirects to the download URL

12.3 Report Technology

Reports are generated using iText/JasperReports with:

- Smart LMS branding header
- Structured table layouts for score breakdowns
- Colour-coded severity indicators
- QR code linking to the online submission view

- Digital timestamp and check ID for audit purposes

---

13. Complete Data Flows & Workflows

13.1 Student Submission Workflow

```
1. Student navigates to /submissions/student/answer/{assignmentId}
2. Frontend decodes JWT → extracts studentId
3. Frontend calls GET /api/projects-and-tasks/project/{id} → loads questions
4. Frontend calls GET /api/submissions/student/{studentId}?assignmentId={id}&status=DRAFT
   → finds existing draft OR calls POST /api/submissions to create new DRAFT
5. Frontend loads existing answers: GET /api/submissions/{submissionId}/answers
6. Student types answer for Question 1

   [3 seconds of inactivity]

7. AI Feedback: POST http://localhost:8083/api/feedback/live
   → { grammarScore, clarityScore, completenessScore, relevanceScore, strengths, improvements,
suggestions }
   → Frontend renders scores in LiveFeedbackPanel

8. Plagiarism (peer + internet): POST http://localhost:8084/api/integrity/checks/realtime
   → { similarityScore, severity, flagged }
   → Frontend renders PlagiarismWarning chip below editor

   [2 more seconds of inactivity (total 5s)]

9. Auto-save: PUT /api/submissions/{id}/answers/{questionId}
   → { answerText, questionText, wordCount, characterCount, studentId }
   → Returns AnswerResponse

10. PATCH /api/submissions/{id}/answers/{questionId}/analysis
    → Persists AI scores + plagiarism result to answers table

11. Student clicks "Submit Assignment"
12. Frontend validates: all questions have answers with ≥ min word count
13. POST /api/submissions/{id}/submit
    → Service validates deadline
    → Service computes ai_generated_mark per question
    → Service computes aggregate aiScore
    → Service transitions status to SUBMITTED
    → Service creates version snapshot (same @Transactional)
14. Frontend navigates to /submissions/student/my-submissions
```

13.2 Instructor Grading Workflow

```
1. After deadline passes, lecturer opens /submissions/lecturer/submissions
2. Page loads: GET /api/submissions?assignmentId={id}&status=SUBMITTED
3. Lecturer clicks "Grade" for a submission
4. Grading page loads:
```

```
   a. GET /api/submissions/{submissionId}
   b. GET /api/submissions/{submissionId}/answers → all answers with AI scores
   c. GET /api/integrity/checks/submission/{submissionId} → plagiarism check
5. For each question:
   a. Lecturer reviews student answer
   b. Sees AI-suggested mark (pre-populated)
   c. Enters lecturer mark + feedback text
   d. Clicks "Save Question"
   e. PATCH /api/submissions/{id}/answers/{qId}/analysis
      → { lecturerMark, lecturerFeedbackText }
6. Lecturer clicks "Finalize Grade"
7. POST /api/submissions/{id}/grade
   → { finalGrade, generalFeedback, questionMarksJson }
   → Status → GRADED
8. Student can now view feedback at /submissions/student/feedback/{id}
```

13.3 Real-Time Analysis Pipeline

```
User keystroke
    │
    ├─ [Every keystroke] Reset all timers
    │
    ├─ [3s after last keystroke] AI Feedback Timer fires
    │       POST /api/feedback/live
    │       ├─ Gibberish check
    │       ├─ Build prompt
    │       ├─ Call HuggingFace API (Zephyr-7B)
    │       ├─ Parse response
    │       ├─ Apply consistency rules
    │       └─ Return LiveFeedbackResponse
    │       Update LiveFeedbackPanel with animated bar fills
    │
    ├─ [3s after last keystroke] Plagiarism Timer fires
    │       POST /api/integrity/checks/realtime
    │       ├─ Peer comparison (up to 200 answers, 4 algorithms)
    │       ├─ Internet search (SerpAPI, 3 results)
    │       ├─ Risk score calculation
    │       ├─ Severity bucketing
    │       └─ WebSocket push if HIGH
    │       Update PlagiarismWarning chip
    │
    └─ [5s after last keystroke] Auto-save Timer fires
            PUT /api/submissions/{id}/answers/{qId}
            → Upsert answer to DB
            PATCH /api/submissions/{id}/answers/{qId}/analysis
            → Persist AI + plagiarism scores
            Update save status: "Saved ✓"
```

13.4 Versioning Workflow

```
Student clicks "Submit"
    │
    POST /api/submissions/{id}/submit
    │
    @Transactional begins
    │
    ├─ Validate deadline (throws if past)
    ├─ Load all answers
    ├─ Compute ai_generated_mark per answer
    ├─ Compute aggregate aiScore for submission
    ├─ Update submission: status=SUBMITTED, submittedAt=now
    ├─ Create version snapshot:
    │       ├─ Find max existing version number
    │       ├─ Idempotency check (existsBySubmissionIdAndVersionNumber)
    │       ├─ Serialize all answers to JSON
    │       └─ Insert SubmissionVersion record
    │
    @Transactional commits (all-or-nothing)
    │
    Return updated Submission
```

---

14. Performance, Scalability & Security

14.1 Performance Optimizations

Optimization	Implementation	Impact
Debounced API calls	3s/5s timers in `useAnswerEditor`	~95% reduction in live API calls vs. per-keystroke
LLM response caching	Caffeine cache (10 min TTL, key = question+answer hash)	Eliminates redundant LLM calls for same content
Async AI feedback	`@Async("feedbackTaskExecutor")` thread pool	Non-blocking Tomcat threads, supports concurrent users
Indexed DB queries	Composite indexes on `(submission_id, question_id)` and `(question_id, last_modified)`	O(log n) upsert and peer lookup
Peer comparison cap	Top 200 answers by recency	Bounded O(n) comparison regardless of class size
Sequential bulk recheck	`for...of + await`	Prevents SerpAPI rate limit violations

14.2 Scalability Considerations

The current architecture supports a single-node deployment. Key bottlenecks and their mitigations:

Bottleneck	Current State	Mitigation Approach

HuggingFace API rate limit	Thread pool of 20 concurrent requests	Add request queue with backpressure; multiple API keys
SerpAPI quota	Sequential processing	Cache internet results by answer content hash
PostgreSQL connection pool	Default HikariCP (10 connections)	Increase pool size; add read replica
WebSocket connections	In-memory session map	Move to Redis-backed session store

14.3 Security

JWT Validation

Every protected endpoint validates the JWT before processing:

```
String token = request.getHeader("Authorization").replace("Bearer ", "");
Map<String, Object> claims = jwtUtils.validateAndExtractClaims(token);
String role = JwtUtils.extractRole(claims);
if (!"lecture".equals(role)) throw new UnauthorizedException("Lecturer role required");
```

SQL Injection Prevention

All database queries use Spring Data JPA's JPQL with named parameters — no raw string SQL concatenation:

```
@Query("SELECT a FROM Answer a WHERE a.submissionId = :submissionId AND a.questionId =
:questionId")
Optional<Answer> findBySubmissionIdAndQuestionId(
    @Param("submissionId") Long submissionId,
    @Param("questionId") String questionId
);
```

Input Length Limits

- `answer_text` : `TEXT` (no DB limit, but LLM is called only for texts ≤ 10,000 characters)
- `question_text` : `VARCHAR(2000)` — enforced at DB level
- `plagiarism_severity` : `VARCHAR(10)` — prevents oversized strings

CORS Configuration

All backend services configure CORS to allow requests only from `http://localhost:3000` in development, with environment-specific overrides for production deployments.

---

15. Testing Strategy

15.1 Backend Testing

Unit Tests

Each service class has a corresponding `*Test.java` class:

Test Class	Coverage Focus

`AnswerServiceTest`	Upsert logic, null handling, mark computation
`SubmissionServiceTest`	State machine, deadline enforcement, aggregate scoring
`LiveFeedbackServiceTest`	Gibberish detection, response parsing, consistency rules
`TextSimilarityServiceTest`	All 4 algorithm correctness with known inputs
`RealtimeCheckServiceTest`	Risk score computation, severity bucketing

Integration Tests

Integration tests start an embedded H2 database and test the full request-response cycle:

```
@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT)
@AutoConfigureTestDatabase(replace = Replace.ANY)
class AnswerControllerIntegrationTest {
    @Test
    void saveAndRetrieveAnswer_roundTrip() {
        // POST /api/submissions/{id}/answers/{qId}
        // GET /api/submissions/{id}/answers
        // Assert: answer text preserved, timestamps set
    }
}
```

Consistency Rule Tests

The scoring consistency tests use parameterized test cases:

```
@ParameterizedTest
@CsvSource({
    "9.5, false, 7.5",   // High score + negative language → capped at 7.5
    "2.0, true, 4.0",    // Low score + positive language → raised to 4.0
    "6.0, false, 6.0"    // Normal case → unchanged
})
void applyConsistencyRules_correctsScores(double input, boolean positiveLanguage, double
expected) { ... }
```

15.2 Frontend Testing

- **Component tests**: Vitest + React Testing Library for `QuestionCard`, `LiveFeedbackPanel`, `PlagiarismWarning`
- **Hook tests**: `useAnswerEditor` debounce timer behavior, state transitions
- **API mock tests**: `msw` (Mock Service Worker) for testing error states and loading states

15.3 Manual Testing Scenarios

Scenario	Steps	Expected Outcome
Auto-save on keystroke	Type in editor, wait 5s	Answer saved, "Saved ✓" shown
AI feedback triggers	Type 10+ words, wait 3s	LiveFeedbackPanel shows scores

Plagiarism detected	Paste peer answer, wait 3s	MEDIUM/HIGH chip shown
Deadline enforcement	Submit after deadline	Error: "Deadline passed"
Version on resubmit	Submit twice	Two versions in timeline
Lecturer grade override	Enter lecturer mark	AI mark preserved, lecturer mark shown separately

---

16. System Integrations

16.1 Projects & Tasks Service (Teammate Integration)

The submission component integrates with the Projects & Tasks service to load assignment questions:

Endpoint	Data Mapped
`GET /api/projects-and-tasks/project/{id}`	`mainTasks[]` → `Question[]` (each title = question text)
`GET /api/projects-and-tasks/task/{id}`	`subtasks[]` → `Question[]` (each title = question text)

Assignment type is preserved: `'project'` for project-based assignments, `'task'` for task-based assignments.

16.2 HuggingFace Inference API

- **Model**: `HuggingFaceH4/zephyr-7b-beta` via Featherless AI routing
- **Call pattern**: POST to HuggingFace text-generation endpoint
- **Timeout**: 120 seconds (configurable)
- **Max tokens**: 500 per response (configurable)
- **Error handling**: On timeout or API error → return a fixed "feedback unavailable" response (silent failure in real-time mode)

16.3 SerpAPI (Google Search)

- **Purpose**: Retrieve internet content for plagiarism comparison
- **Queries**: 3 results per answer text query
- **Rate limit handling**: Sequential processing, configurable delay between batch checks
- **Error handling**: If SerpAPI call fails → internet similarity = 0.0, peer comparison still runs

16.4 PostgreSQL

- **Version**: PostgreSQL 15 (via Docker)
- **Connection**: HikariCP connection pool
- **Schema management**: Hibernate `ddl-auto=update` (creates/alters tables on startup)
- **Schema isolation**: `submission_schema` and `integrity_schema` namespaces prevent cross-service table conflicts

---

17. Technical Challenges & Innovations

17.1 Challenge: LLM Output Consistency

Problem: The Zephyr-7B model occasionally returns scores that contradict its own narrative feedback. For example, giving grammar a score of 9.2 while simultaneously listing "Poor sentence structure" as an improvement point.

Solution: The three-rule consistency enforcement system (Section 6.5) post-processes all LLM outputs to detect and correct internal contradictions using simple keyword matching and score range heuristics. This required careful calibration to avoid over-correcting genuinely nuanced assessments.

17.2 Challenge: Hibernate DDL Migration on Non-Empty Tables

Problem: When new columns with `NOT NULL` constraints are added to an entity class, Hibernate's `ddl-auto=update` generates `ALTER TABLE ADD COLUMN ... NOT NULL` SQL, which PostgreSQL rejects when rows already exist (no DEFAULT value).

Solution: New columns that are semantically optional (like `created_at`) are declared without `nullable = false` , allowing the migration to add a nullable column. The `@CreationTimestamp` annotation still populates the column correctly for all new rows.

17.3 Challenge: Atomic Version Snapshotting

Problem: Version snapshots must be created at exactly the same instant as the submission state change. If snapshots are created in a separate transaction (or a separate service), a crash between the two operations leaves the system in an inconsistent state (submission is SUBMITTED but has no version).

Solution: The version snapshot creation is performed within the same `@Transactional` method as the submission state change. If snapshot creation fails, the entire transaction rolls back and the submission remains in DRAFT state — no partial state is ever committed.

17.4 Innovation: Ensemble Plagiarism Detection

Rather than relying on a single similarity algorithm, the system runs four distinct algorithms in parallel and takes the maximum score. This ensemble approach:

- Catches phrase-level plagiarism (N-gram Dice) missed by term-frequency methods
- Catches structural reordering (TF-IDF cosine) missed by exact-match methods
- Reduces false negatives compared to any single algorithm
- Maintains bounded performance by running on a capped peer set (200 answers)

17.5 Innovation: Live Projected Grade Pill

As the student types, a "projected grade" pill is displayed in the editor header, computed from the current AI feedback scores using the weighted formula:

```
const projectedGrade = liveFeedback
    ? Math.round(
        (liveFeedback.relevanceScore * 0.40 +
         liveFeedback.completenessScore * 0.30 +
         liveFeedback.clarityScore * 0.15 +
         liveFeedback.grammarScore * 0.15) * maxPoints / 10.0
      )
    : null;
```

This gives students a real-time indication of their standing before submission, incentivizing improvement rather than mere completion.

---

18. Limitations & Future Work

18.1 Current Limitations

Limitation	Description	Impact
Single-node deployment	No horizontal scaling	Limited concurrent user capacity
Synchronous inter-service calls	RestTemplate blocking calls	Service failures cascade
LLM dependency	Requires HuggingFace API availability	Feedback unavailable during outages
SerpAPI quota	Limited Google searches per day	Internet plagiarism checking may be rate-limited
No real-time collaboration	Single-editor model	Cannot support group submissions natively
In-memory WebSocket sessions	Not Redis-backed	WebSocket state lost on service restart
File submission not analysed	Only text answers get AI/plagiarism treatment	Students can bypass analysis via file upload

18.2 Planned Improvements

1. **Event-driven architecture**: Replace synchronous RestTemplate calls with Apache Kafka topics for AI feedback and plagiarism results, improving fault isolation.
2. **Horizontal scaling**: Containerise all services for Kubernetes deployment with HPA (Horizontal Pod Autoscaling) based on request queue depth.
3. **Feedback personalisation**: Track student's improvement trajectory across submissions and tailor feedback to specifically address recurring weaknesses.
4. **Code submission analysis**: Extend the plagiarism engine to support code submissions using Abstract Syntax Tree (AST) comparison via JPlag.
5. **Offline draft support**: Implement Service Worker-based local caching so students can write answers without an internet connection, syncing when reconnected.
6. **Multi-language support**: Extend AI feedback to support non-English submissions by switching to a multilingual LLM.
7. **Redis-backed WebSocket sessions**: Move WebSocket session state to Redis for multi-node WebSocket support.

---

19. Technology Stack

19.1 Backend

Component	Technology	Version	Purpose
Application Framework	Spring Boot	3.x	REST APIs, DI, lifecycle

ORM	Spring Data JPA / Hibernate	6.x	Database access
Database Driver	PostgreSQL JDBC	42.x	PostgreSQL connectivity
Connection Pool	HikariCP	5.x	DB connection management
Async Processing	Spring `@Async`	6.x	Non-blocking AI calls
Caching	Caffeine	3.x	In-memory LRU cache
WebSocket	Spring WebSocket (STOMP)	6.x	Real-time notifications
Math Library	Apache Commons Math	3.x	TF-IDF vector operations
HTTP Client	Spring RestTemplate	6.x	Inter-service calls
PDF Generation	iText / JasperReports	7.x	Report generation
Code Reduction	Lombok	1.18.x	Boilerplate elimination
Build Tool	Maven	3.9.x	Dependency management
Container	Docker	25.x	Deployment

19.2 Frontend

Component	Technology	Version	Purpose
Framework	Next.js	16	SSR + routing
UI Library	React	19	Component model
Language	TypeScript	5.x	Type safety
Styling	Tailwind CSS	4	Utility-first CSS
Icons	Lucide React	Latest	Consistent icon set
HTTP	Browser Fetch API	—	API calls
State	React Hooks	—	Local state management
Build	Turbopack (Next.js)	—	Fast dev builds

19.3 Infrastructure

Component	Technology	Purpose
Database	PostgreSQL 15	Primary data store
Cache	Redis	Session store, queue (planned)
Container Orchestration	Docker Compose	Local multi-service setup
Search API	SerpAPI	Internet similarity queries
LLM API	HuggingFace Inference / Featherless	AI feedback generation

---

20. API Reference

20.1 Submission Management Service (Port 8081)

Submissions

Method	Endpoint	Request Body	Response	Description
POST	`/api/submissions`	`{ assignmentId, studentId, submissionType }`	`Submission`	Create draft submission
GET	`/api/submissions/{id}`	—	`Submission`	Get submission by ID
GET	`/api/submissions/student/{studentId}`	—	`Submission[]`	Get student's submissions
POST	`/api/submissions/{id}/submit`	—	`Submission`	Formally submit (DRAFT → SUBMITTED)
POST	`/api/submissions/{id}/grade`	`{ finalGrade, feedback, questionMarksJson }`	`Submission`	Set final grade (SUBMITTED → GRADED)

Answers

Method	Endpoint	Request Body	Response
PUT	`/api/submissions/{id}/answers/{questionId}`	`SaveAnswerRequest`	`AnswerResponse`
GET	`/api/submissions/{id}/answers`	—	`AnswerResponse[]`
GET	`/api/submissions/{id}/answers/{questionId}`	—	`AnswerResponse`
PATCH	`/api/submissions/{id}/answers/{questionId}/analysis`	`SaveAnswerAnalysisRequest`	`AnswerResponse`

Versions

Method	Endpoint	Response	Description
GET	`/api/submissions/{id}/versions`	`SubmissionVersion[]`	Get all versions
GET	`/api/submissions/{id}/versions/{versionNumber}`	`SubmissionVersion`	Get specific version

GET	`/api/submissions/{id}/versions/compare`	`VersionDiff`	Diff two versions

20.2 Feedback Service (Port 8083)

Method	Endpoint	Request Body	Response	Description
POST	`/api/feedback/live`	`LiveFeedbackRequest`	`LiveFeedbackResponse`	Generate real-time AI feedback

LiveFeedbackRequest:

```
{
  "questionText": "string",
  "answerText": "string",
  "studentId": "string"
}
```

LiveFeedbackResponse:

```
{
  "grammarScore": 8.2,
  "clarityScore": 7.5,
  "completenessScore": 9.0,
  "relevanceScore": 8.8,
  "strengths": ["string"],
  "improvements": ["string"],
  "suggestions": ["string"],
  "processingTimeMs": 2840
}
```

20.3 Integrity Monitoring Service (Port 8084)

Method	Endpoint	Request Body	Response
POST	`/api/integrity/checks`	`IntegrityCheckRequest`	`PlagiarismCheck`
POST	`/api/integrity/checks/realtime`	`RealtimeCheckRequest`	`RealtimeCheckResponse`
GET	`/api/integrity/checks/{id}`	—	`PlagiarismCheck`
GET	`/api/integrity/checks/flagged`	—	`PlagiarismCheck[]`
GET	`/api/integrity/checks/assignment/{id}`	—	`PlagiarismCheck[]`

GET	`/api/integrity/checks/submission/{id}`	—	`PlagiarismCheck`
GET	`/api/integrity/reports/{submissionId}/plagiarism`	—	PDF (async)
GET	`/api/integrity/reports/{submissionId}/feedback`	—	PDF (async)

End of Documentation

Document Information:

- Total Sections: 20
- Component: Smart LMS Submission Management (IT22586766)
- Last Updated: April 2026
- Classification: Technical Reference / Research Documentation